

Architecture

Diagrams of the live system — request flow, caching, and abuse protection.

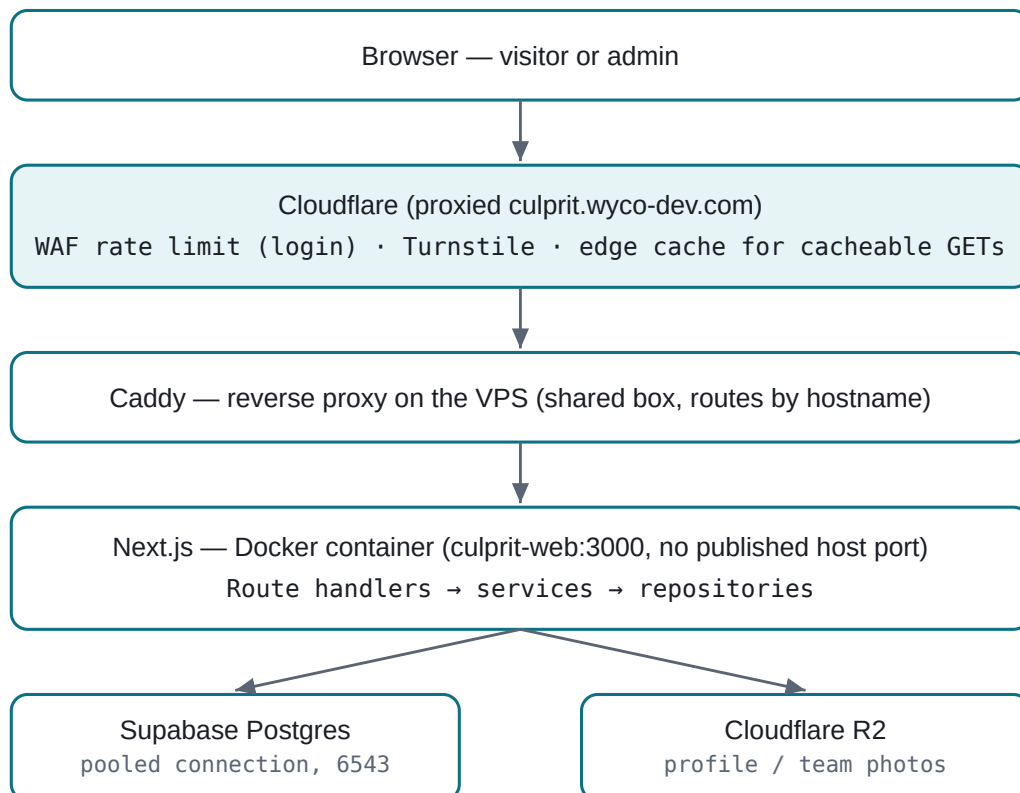
Hosting: self-hosted VPS, Docker

CDN/Edge: Cloudflare

Status: Live

01 System Architecture

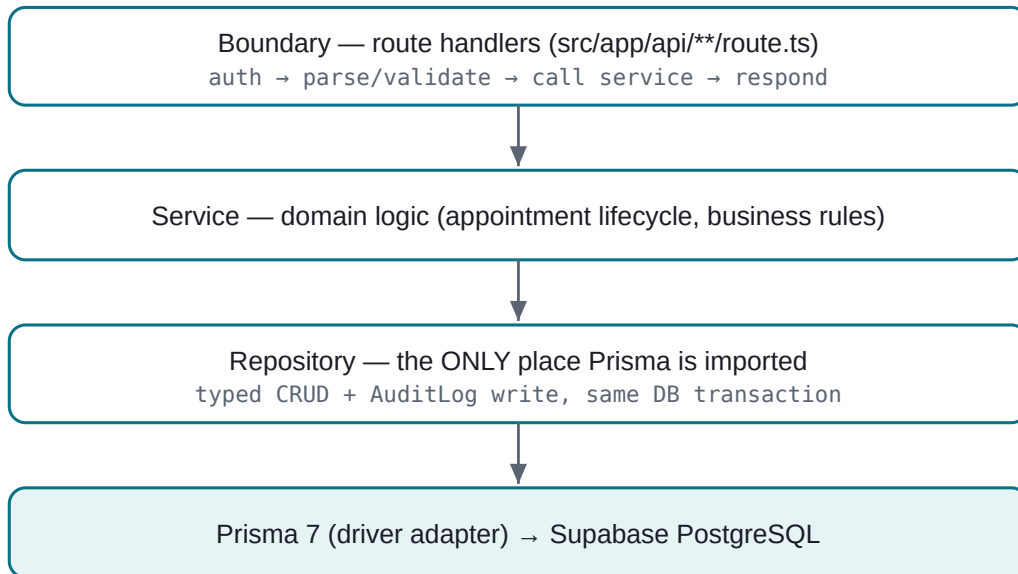
Every request — visitor or admin — enters through the same front door. There is no separate API host or second deployment: one Next.js container handles pages and API routes alike.



Calendly is embedded client-side only — the browser talks to Calendly directly; no server-side hop.

02 Application Layers

Inside the Next.js container, data flows one direction — each layer depends only on the one below it. Prisma is imported in exactly one place: the repository layer.

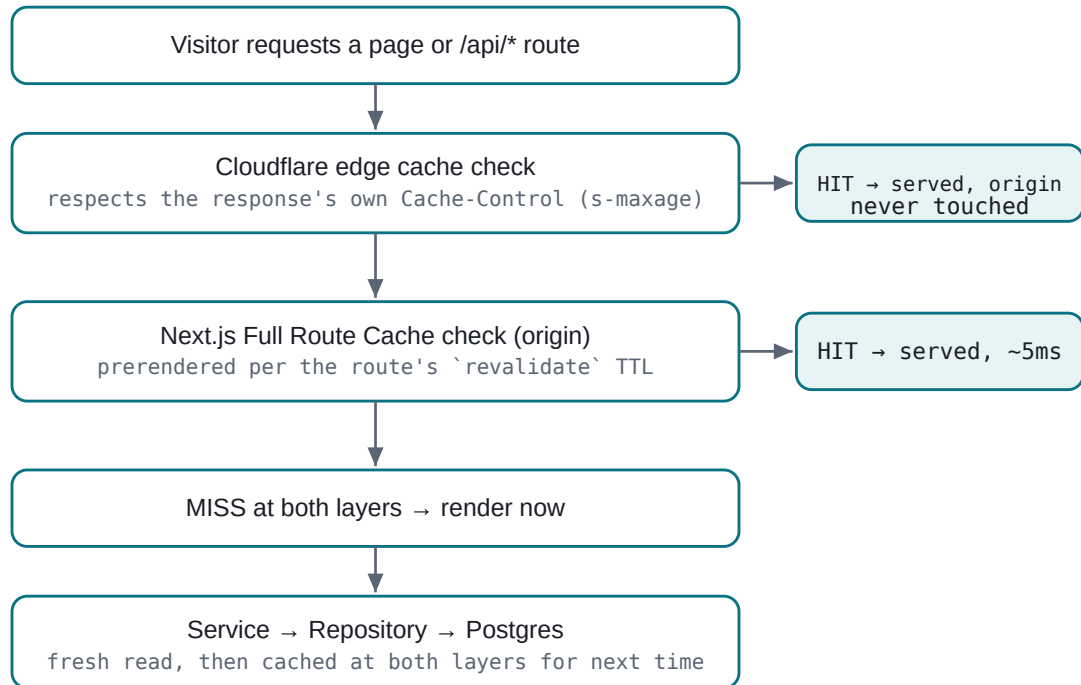


The audit-log write happens inside the repository, atomic with the mutation — not in the service layer.

Modules (src/modules/*): auth , profile , research , publications , research-groups , appointments , settings , integrations (Calendly embed, R2 storage, Turnstile, rate limiter, YouTube), shared (Result/error types, logger, API envelope, Prisma singleton).

03 How a Visitor Gets Data

A public page or API route can be answered from three different places, checked in order. Only a full cache miss at every layer actually touches the database.



CDN (Cloudflare edge) is the outermost layer; Next's Full Route Cache is the innermost, on the origin itself.

04 Cache Layers & TTLs

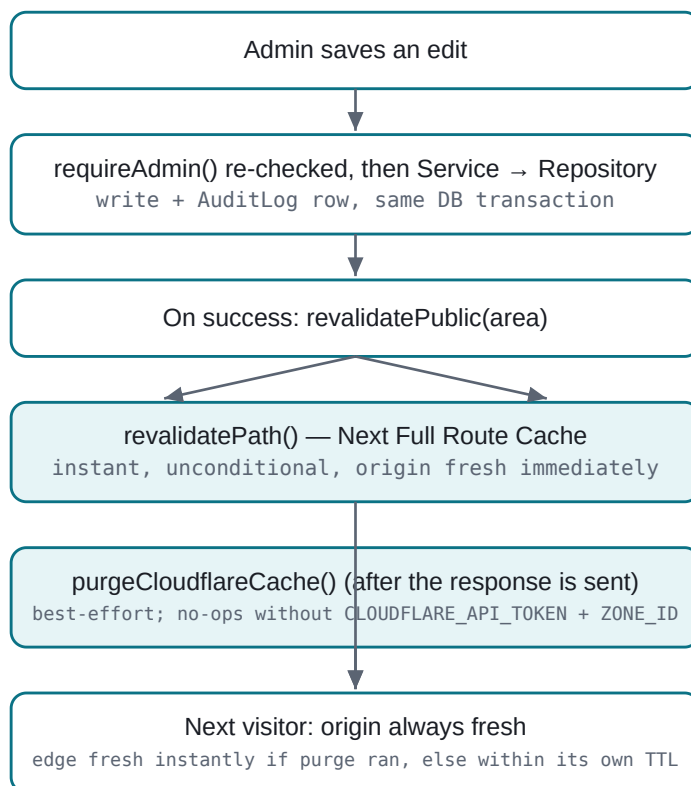
Real numbers from the running config — the browser and edge intentionally get different TTLs (`max-age` vs `s-maxage`), so a stale browser tab catches up sooner than the shared edge cache does.

CONTENT	BROWSER (MAX-AGE)	EDGE / ORIGIN (S-MAXAGE)
Static assets (JS, CSS, images)	1 year	1 year
Public pages (About, Research, etc. — Full Route Cache)	—	1 hour
Research / publications / profile / team-members API	5 min	1 hour
Upcoming-events API	1 min	5 min
Anything behind admin login (<code>/api/auth/*</code> , <code>/api/admin/*</code>)	no-store	no-store

Events gets a shorter TTL on purpose — an event can slip from "upcoming" to "past" while nobody's looking, so a shorter window limits how long a stale, already-past event can keep showing.

05 Admin: No Cache, Invalidate on Save

Admin pages and API routes are never cached — `requireAdmin()` re-checks the session on every single request, and the response is marked `private, no-store` explicitly (defense-in-depth, not just the absence of a header). A successful save then pushes freshness back out to visitors immediately.

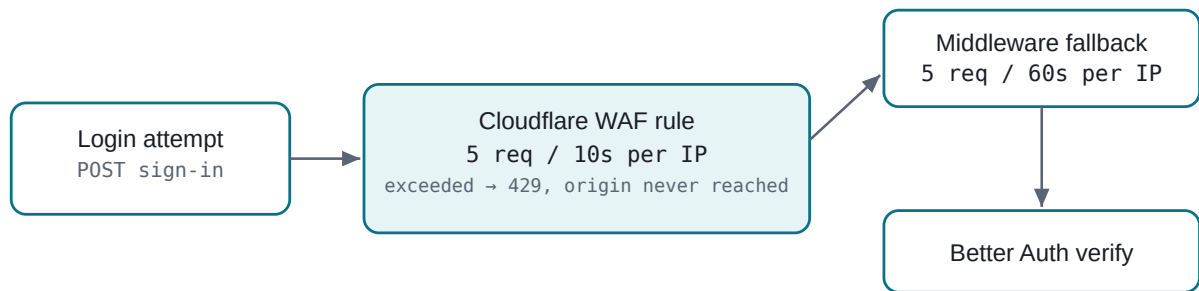


Origin freshness never depends on the Cloudflare token — it's a pure optimization on top of an already-correct origin.

06 Rate Limiting & Turnstile

Login brute force

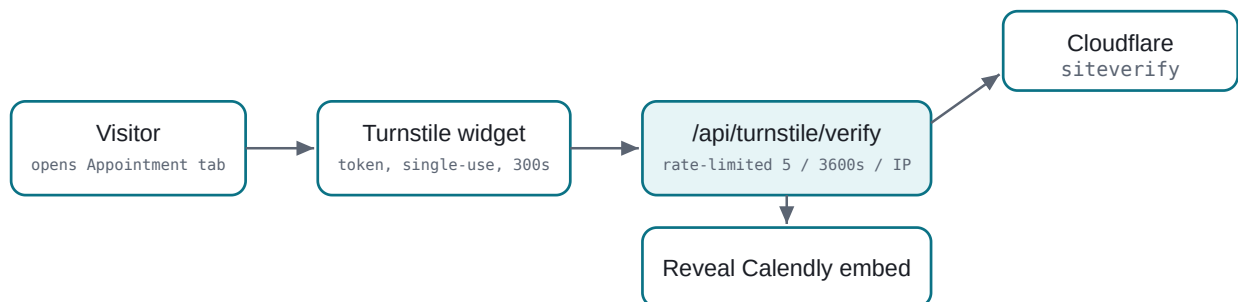
Two layers, because Cloudflare's Free plan allows exactly one WAF rate-limiting rule per zone — it's spent on the highest-severity target.



The middleware fallback (in-memory, per-instance) also covers mutating `/api/admin/*` at 30 req/60s – a stolen session cookie can't be hammered, though Better Auth's own session check is the real gate there.

Turnstile — gating the Calendly embed

Calendly's free plan has no bot protection and one bookable event type — Turnstile decides whether the booking widget appears at all.



The client-side widget proves nothing by itself – only a confirmed server-side siteverify call reveals the booking calendar. Failure keeps it hidden (fail-closed).